

GPT: Generative Pre-trained Transformer

Lecture 5 — Building LLMs from Scratch

Introduction

Hello everyone, welcome to this lecture in the *Building Large Language Models from Scratch* series. This is Lecture 5, and today we are going to look in detail at GPT, or Generative Pre-trained Transformer. We are going to see how the versions of GPT evolved, how the different papers on GPT evolved, what is the progression from Transformers to GPT to GPT-2 to GPT-3, and finally to GPT-4 where we are at right now.

In the previous lectures, we have looked at Transformers; we looked at a simplified architecture for Transformer, and we also saw the difference between BERT and GPT model; we saw what is the meaning of an encoder, what is the meaning of a decoder, etc. If you have not seen the previous lecture I would highly encourage you to go through that; if not, it is totally fine—the way I have designed this lecture is such that it is self-contained and you will understand all the concepts which I am explaining in this lecture.

So one of the key concepts which we are also going to look at today is zero-shot versus few-shot learning. But before coming to that, let me first take you through the progression of research papers from Transformers to GPT to GPT-2, and then finally to GPT-3 and then GPT-4.

The History: From Transformers to GPT-4

Let us deep-dive a bit into history.

2017: “Attention Is All You Need” (Transformers)

The first paper which was released which really started the work on GPT is “Transformers”. This paper was released in 2017, and this paper was titled *Attention Is All You Need*. The major breakthrough of this paper is introducing the self-attention mechanism, where you capture the long-range dependencies in a sentence, allowing you to do a much better job at predicting the next word in a sentence. It represented a significant advancement compared to Recurrent Neural Networks (RNNs) and Long Short-Term Memory networks (LSTMs). Transformers, which were introduced in this paper, really changed everything.

The original Transformer architecture looked something like this: it had an encoder and it had a decoder. But the GPT architecture, which came after this paper, did not have an encoder—the GPT architecture which was developed only has a decoder.

2018: Generative Pre-training (GPT-1)

This was the paper which came out after the Transformers paper, and it introduced the concept of generative pre-training. The main idea of this concept is unsupervised learning. Basically, they said that natural language processing as such up till this point had been mostly supervised learning; so they were saying that labeled data for learning is scarce, which means it is not easily available, making it challenging for trained models to perform adequately.

They wrote:

“We demonstrate that large gains on these tasks can be realized by generative pre-training of a language model on a diverse corpus of unlabeled text.”

Two words are important here: **generative pre-training** and **unlabeled text**. What is done here is that the text which is used is not labeled. So let us say you have given a sentence, right? And you use that sentence itself as the training data, and the next word prediction which is in that sentence itself as the testing data. So everything is self-contained, and you do not need to provide labels.

OpenAI also released a blog about this on June 11, 2018: *Improving Language Understanding with Unsupervised Learning*. Their main claim was:

“We have obtained state-of-the-art results on language tasks with a scalable system. Our approach is a combination of two ideas: Transformers and unsupervised pre-training.”

2019: GPT-2

Then in 2019, just the next year, came one more paper which is called *Language Models are Unsupervised Multitask Learners*. What they basically did is they just took more amount of data than was used in the earlier paper, and they also used a generative pre-trained network. They showed four types of generative pre-trained networks: GPT-2 Small (117M parameters), GPT-2 Medium (345M), GPT-2 Large (762M), and GPT-2 Extra Large (1.5B parameters).

2020: GPT-3

Then in 2020 came the real boss, which was GPT-3. GPT-3 had 175 billion parameters. They released a number of versions of GPT-3: small, medium, large, extra large—a version with 2.7 billion parameters, a version with 6.7 billion parameters. But there was one specific version which was released which had 175 billion parameters, which was GPT-3. And when people started exploring this model, they could really see that it is amazing. It could do so many things: although it was just trained to predict the next word, it can do a number of other things like translation, sentiment analysis, answering questions, answering multiple-choice questions, emotional recognition—it can do so many things. This was a huge model: 175 billion parameters. People had not seen language models of this size.

GPT-3.5 and GPT-4

Two years after this came GPT-3.5, which became commercially viral. Everyone started using it and saw how good it was. And right now, GPT-4 is where we are today.

Year	Milestone
2017	Transformers (<i>Attention Is All You Need</i>)
2018	GPT-1 (Generative Pre-training)
2019	GPT-2 (1.5B parameters)
2020	GPT-3 (175B parameters, <i>Few-Shot Learner</i>)
—	GPT-3.5 (ChatGPT, commercially viral)
2024	GPT-4 (current state-of-the-art)

Many people do not know the difference between Transformers and GPT. GPT essentially borrows from the Transformer architecture, but it is a bit different in that it does not have the encoder block.

Zero-Shot vs. Few-Shot Learning

The next thing which I want to cover today is the difference between zero-shot and few-shot learning.

Zero-shot learning is basically the ability to generalize to completely unseen tasks without any prior specific examples. For example, the prompt can be: “Translate English to French: cheese → ?” If the model is able to do that without any supporting examples, that is an example of zero-shot learning.

One-shot learning means that in addition to the task description, the model also sees a single example of the task. So you say: “Sea otter → loutre de mer. Now translate: cheese → ?”

Few-shot learning means the model sees a few examples of the task. For example:

- Sea otter → loutre de mer
- Peppermint → menthe poivrée
- Giraffe → girafe

Then translate: “cheese → ?”

Let us see the claim of the GPT-3 authors:

“We train GPT-3, an autoregressive language model with 175 billion parameters, $10\times$ more than any previous non-sparse language model, and test its performance in the few-shot setting. GPT-3 achieves strong performance on translation, question-answering, as well as several tasks that require on-the-fly reasoning or domain adaptation, such as unscrambling words, using a novel word in a sentence, or performing three-digit arithmetic.”

The paper basically implied that GPT-3 was a few-shot learner. But what about GPT-4? Is it a zero-shot learner or a few-shot learner? Because it seems that we do not need to give it examples—it just does many things on its own.

GPT-4 itself says: “I am a few-shot learner. This means I can understand and perform tasks better with a few examples. While I can handle many tasks without prior examples (zero-shot learning), providing examples helps me generate more accurate responses.”

So GPT-4 is both a zero-shot learner and a few-shot learner, but to get more accurate responses, you need to probably provide it a few examples.

Utilizing Large Datasets: GPT-3’s Training Data

Let us look at the dataset which GPT-3 used. The model was 175 billion parameters, but let us see the data on which the model is trained:

Dataset	Tokens	Weight
Common Crawl	410B tokens	60%
WebText2	19B tokens	22%
Books1	12B tokens	8%
Books2	55B tokens	8%
Wikipedia	3B tokens	3%

The total number of tokens on which GPT-3 is trained is approximately 300 billion tokens. Think of that for a while: 300 billion tokens—that is a huge amount of data, and it would need a huge amount of compute power and cost.

The Cost of Pre-training GPT-3

One more key thing to keep in mind is that the total pre-training cost for GPT-3 is **\$4.6 million**. Training GPT-3 is not easy. Pre-training is not easy. You need a huge amount of computer power and you need a huge amount of data.

Pre-trained models are also called as the base models or foundational models, which can be used later for fine-tuning. The process of fine-tuning means that if you are a banking company or an airline company or an educational company which wants to use GPT-3 but you also want the output to be specific to your dataset, then you need to fine-tune the model further on labeled data which belongs to your organization.

GPT Architecture

The GPT architecture consists of only a decoder block. The original Transformer consists of encoder as well as decoder. GPT-3 is a scaled-up version of the original Transformer model which was implemented on a larger dataset. GPT-3 has 175 billion parameters.

Auto-regressive and Unsupervised Nature

Why are GPT models called auto-regressive models, and why do they come under the category of unsupervised learning?

GPT models are simply trained on next-word prediction tasks. For example, consider the sentence: “The lion is in the ____” and you want to predict the next word—then you predict that it is going to be “jungle.” This is all what GPT models are trained for.

Here is how the training process actually looks like. Let us say we have the phrase “Second Law of Robotics: A robot must.” The training proceeds in iterations:

1. Input: “Second Law of Robotics:” → Output: “A”
2. Input: “Second Law of Robotics: A” → Output: “robot”
3. Input: “Second Law of Robotics: A robot” → Output: “must”
4. And so on...

At every stage of this training process, the sentence is broken up into input and output. The input consists of some words, but the output is always one word. We do not give any labels here. The sentence itself is broken into two parts: the first part and the second part. The second part is where we have to predict the next word. That is why it is called unsupervised learning.

Auto-regressive Property

The previous output is used as the input for the future prediction. Let us go over this again:

- First input: “Second Law of Robotics:” → Output: “A”
- This “A” becomes an input to the next sentence: “Second Law of Robotics: A” → Next word: “robot”
- This “robot” becomes an input to the next sentence

That is why the model is also called auto-regressive.

GPT Decoder-Only Architecture

Compared to the original Transformer architecture, the GPT architecture is actually simpler. The GPT architecture only has the decoder block; it does not have the encoder block. In the original Transformer we had six encoder-decoder blocks. In the GPT-3 architecture, on the other hand, we have **96 Transformer layers** and 175 billion parameters.

The way GPT works is in iterations:

- **Iteration 1:** Input “This” → Pre-processing (token IDs) → Decoder → Output layer → Predict “is”

- **Iteration 2:** Input “This is” → Pre-processing → Decoder → Output layer → Predict “an”
- **Iteration 3:** Input “This is an” → Pre-processing → Decoder → Output layer → Predict “example”

In each iteration we predict the next word, and then the prediction informs the input of the next iteration. That is why it is unsupervised and auto-regressive. The encoder block is not present in the GPT architecture—only the decoder block is present.

Emergent Behavior

GPT is trained only for the next word prediction, right? GPT is trained only for the next word prediction. But it is actually quite awesome and amazing that even though GPT is only trained to predict the next word, it can perform so many other tasks such as language translation.

Even though GPT was not trained to do translation tasks, GPT-4 can still do the translation tasks. GPT was not trained to do MCQ generation, but if you ask GPT to provide multiple-choice questions, it will generate them.

This is called **emergent behavior**. The formal definition of emergent behavior is:

“The ability of a model to perform tasks that the model was not explicitly trained to perform.”

This was also mentioned in one of the blogs of OpenAI, where they say:

“We noticed that we can use the underlying language model to begin to perform tasks without ever training on them. For example, performance on tasks like picking the right answer to a multiple-choice question steadily increases as the underlying language model improves.”

This was very surprising to researchers also, because it was only trained to do the next-word tasks, so how can it develop these many capabilities? This still remains an open question. This is an area of active research, such as exploring emergent behavior in LLMs. If any of you are considering looking for research topics, emergent behavior might be a great topic to start your research.

Recap

Let me do a quick recap of what all we have covered:

1. **History:** We saw that the first paper which was introduced in 2017 is *Attention Is All You Need*, which incorporated the Transformer architecture.
2. **Generative Pre-training (GPT):** The architecture is a bit different than Transformer—it uses only decoder, no encoder. It shows two main things: first, it is unsupervised; second, it is auto-regressive on unlabeled data.
3. **GPT-2 (2019):** Four models of GPT-2 were released: 117M, 345M, 762M, and 1.5B parameters.
4. **GPT-3 (2020):** *Language Models are Few-Shot Learners*—175 billion parameters, the largest anyone had ever seen up till that point.
5. **Zero-shot vs. Few-shot:** Zero-shot means no examples; few-shot means a few supplementary examples. GPT-4 is both, but it is more accurate at few-shot learning.
6. **GPT-3 Data:** 300 billion tokens total; pre-training cost \$4.6 million.

7. **GPT Architecture:** Only the decoder; works in iterations; the output of one iteration is fed as input to the next iteration (auto-regressive); the sentence itself is used to make the label which is the next word prediction (unsupervised).
8. **Fine-tuning:** Training on a much narrower and specific dataset to improve performance; usually needed in production-level tasks.
9. **Emergent Behavior:** The ability of the model to perform tasks that it was not explicitly trained to perform—an active research topic.

In the next lecture, we will look at stages of building an LLM, and then we will start coding directly from the data pre-processing.

Thank you so much, everyone, for sticking with me until this point. We have covered five lectures so far, and in all of them I have tried to make them as detailed as possible and as much from a basics approach as possible. Look forward to seeing you in the next video.